

dPrompt Structure and Strategic Reasoning in Large Language Models:

An Empirical Study Using Tic-Tac-Toe as a Benchmark

Abstract

This study investigates how prompt structure influences the strategic reasoning capabilities of large language models (LLMs) using Tic-Tac-Toe, a solved two-player game, as a controlled benchmark. Ten LLMs spanning commercial and open-source architectures were evaluated across 190 games played against a single human opponent who played for optimal outcomes. Six models advanced to a second testing stage in which each played ten games under a structured prompt specifying board representation and explicit win-condition checking, and ten games under an unstructured prompt providing only minimal instruction. Performance was measured through a composite "best outcome" metric that credits wins regardless of move order and draws when the model moved second, alongside standard win, loss, and draw rates, average game length, blunder counts, and blunder timing. Three findings stand out. First, structured prompting did not produce a uniform performance improvement: four of six Stage 2 models achieved equal or higher best outcome rates under unstructured prompts, one performed identically under both, and only GPT OSS 120b showed a clear advantage from structured prompting. Second, every Stage 2 model that won any games performed substantially better when moving second than when moving first, a pattern consistent with the best outcome metric being easier to satisfy from the second-mover position and with reactive defense being a tactically simpler task than first-mover offense. Third, 80–100 percent of model blunders occurred in the middle phase of the game (turns 4–6), consistent with a depth-limited lookahead hypothesis. These findings suggest that the relationship between prompt structure and strategic reasoning performance is model-dependent and that simple benchmarks like Tic-Tac-Toe continue to expose specific and diagnosable weaknesses in LLM middle-game planning.

1. Introduction

Large language models have demonstrated impressive capabilities across a wide range of natural language tasks, yet their performance on problems requiring explicit logical reasoning remains inconsistent. Strategic games offer a useful lens for examining this inconsistency because the correctness of a move can be verified objectively, the search space is bounded, and the task cannot be solved through pattern-matching on natural language alone. Tic-Tac-Toe is particularly well-suited as a benchmark for this purpose. The game is solved: with perfect play from both participants, every game ends in a draw. The state space is small enough that a human evaluator can verify any claimed win condition in seconds, and individual games conclude within nine moves. These properties make repeated testing feasible and establish a clear ground truth against which model behavior can be measured.

The present study examines one central question: does the structure of the prompt affect how well an LLM plays Tic-Tac-Toe? Prior work on prompt engineering has generally found that explicit, decomposed instructions improve model performance on complex reasoning problems, but these findings derive largely from benchmarks that test factual recall, arithmetic, or code generation. Short-horizon strategic games demand a different combination of capabilities, including persistent state tracking across turns, forward-looking threat evaluation, and recognition of terminal states. It is not obvious that prompt structuring techniques that succeed in other domains will transfer to this setting. This paper contributes empirical evidence on that question by comparing paired structured and unstructured prompt conditions across six representative LLMs and analyzing the resulting move-level data for systematic patterns in opening play, first-mover outcomes, and blunder timing.

2. Methods

2.1 Model Selection

Ten models were selected for an initial screening stage: ChatGPT Instant, ChatGPT Thinking, Gemini 3 Thinking, Claude Opus 4.5, Claude Opus 4.6, Claude Opus 4.6 Thinking, Llama 4 Scout, DeepSeek R1, GPT OSS 120b, and Qwen3 Coder Next. The set spans commercial products (ChatGPT, Gemini, Claude) and open-source models running on Indiana University servers (Llama 4 Scout, DeepSeek R1, GPT OSS 120b, Qwen3 Coder Next). Models were chosen to represent both general-purpose and reasoning-oriented variants, enabling comparison between models of similar base architectures that differ in their reasoning mode. After screening, six models were advanced to Stage 2 for extended testing: ChatGPT Thinking, Claude Opus 4.6, Claude Opus 4.6 Thinking, Llama 4 Scout, DeepSeek R1, and GPT OSS 120b.

2.2 Prompt Conditions

Two prompt conditions were defined. The unstructured condition provided minimal instruction, typically a single sentence asking the model to play Tic-Tac-Toe. This prompt leaves board representation, turn order, win-condition checking, and level of play effort unspecified, requiring the model to supply these elements on its own. The structured condition provided a detailed procedural specification: a coordinate system for the board, an instruction to reconstruct the board from the full move list before each turn, an explicit enumeration of horizontal, vertical, and diagonal win checks, and a directive to stop play immediately upon detecting a win. The structured prompt also stated an explicit performance goal of winning with perfect play.

Because the study investigates which prompting approaches each model responds to most effectively rather than comparing verbatim prompts across models, minor wording variations were retained when a particular prompt failed to elicit engaged play from a given model. All six Stage 2 models received variants of the same two template prompts, and the complete verbatim prompts for every model appear in Appendix A. The implications of this design choice are discussed in Section 5.

2.3 Opponent

A single human opponent played every game, attempting optimal or near-optimal play throughout. The opponent's strategy was to win or force a draw in each game. When models produced identical move sequences across consecutive games, the opponent occasionally deviated from their prior line in order to test whether the model would adapt its play; models generally did not adapt unless forced to by the opponent selecting a square the model would otherwise have taken. Against weaker models, the opponent was frequently able to play three consecutive forcing moves without meaningful resistance. Against stronger models, the opponent focused on establishing forks (double-threat configurations) and observing whether the model could defend them; the best models consistently forced draws and occasionally capitalized on intentional opponent blunders designed to test win-recognition behavior. The fact that the opponent played for best outcomes means that model losses should be interpreted as genuine tactical failures rather than artifacts of an adversarial human opponent.

2.4 Experimental Procedure

Each Stage 2 model played ten games under the structured prompt and ten games under the unstructured prompt, for a total of twenty games per model. First-mover assignment was recorded for each game but not controlled. For every game the following parameters were logged: outcome (user win, LLM win, or draw), party responsible for the decisive blunder, first mover, prompt type, wall-clock game duration, total turn count, and the turn range in which the blunder occurred (turns 1–3, 4–6, or 7–9). Move-by-move board state was recorded on a separate worksheet using a 1–9 square labeling in which 1 is the top-left, 5 is the center, and 9 is the bottom-right square, or equivalent (row, column) coordinates. This enabled later analysis of recurring positional patterns and opening-move distributions.

2.5 Performance Metrics

Raw win, loss, and draw rates were calculated for each model under each prompt condition. Because Tic-Tac-Toe is a solved game in which the second mover can guarantee at most a draw, these raw rates do not fully capture strategic competence. A composite best outcome metric was therefore defined: a game counts toward the best outcome rate if the model won (regardless of first-mover status) or if the model achieved a draw while moving second. This metric more accurately reflects whether the model achieved the best outcome available to it. Average total turns per game was also recorded as a proxy for strategic depth, under the assumption that stronger play tends to extend games closer to the nine-turn maximum by blocking threats and avoiding early losses. Blunder counts and blunder-turn ranges provided additional granularity on where in the game failures concentrated.

3. Results

3.1 Overall Performance

Table 1 summarizes the pooled performance of each model across all games played. Claude Opus 4.6 Thinking achieved the highest best outcome rate at 60 percent, followed by Gemini 3 Thinking at 50 percent. Llama 4 Scout failed to achieve a best outcome in any game, losing 100 percent of the twenty games in its sample. The remaining models clustered between 10 and 40 percent best outcome rates. Average turns per game correlated with overall competence: the top-performing models (Claude Opus 4.6 Thinking, Gemini 3 Thinking, DeepSeek R1, Claude Opus 4.5, Claude Opus 4.6) averaged 7.85 turns or longer, while the weakest performer (Llama 4 Scout) averaged only 5.0 turns. This pattern is consistent with the interpretation that stronger play prolongs the game by blocking immediate threats, whereas weaker play collapses quickly once a fork or open two-in-a-row emerges.

Model	Games	Best Outcome %	Win %	Loss %	Draw %	Avg Turns
ChatGPT Instant	20	5%	0%	90%	10%	6.6
ChatGPT Thinking	20	20%	0%	65%	35%	7.25
Gemini 3 Thinking	10	50%	20%	20%	60%	8.10
Claude Opus 4.5	10	40%	0%	50%	50%	8.00
Claude Opus 4.6	20	30%	0%	45%	55%	7.85
Claude Opus 4.6 Thinking	30	60%	13%	13%	73%	8.30
Llama 4 Scout	20	0%	0%	100%	0%	5.00
DeepSeek R1	20	40%	5%	20%	75%	8.00
GPT OSS 120b	30	40%	3%	30%	67%	7.90
Qwen3 Coder Next	10	10%	10%	80%	10%	6.10

Table 1. Pooled performance across all games played for each model in the study.

3.2 Structured Versus Unstructured Prompts

Table 2 compares performance under the two prompt conditions for the six Stage 2 models. The results show no uniform advantage for structured prompting. Three models (ChatGPT Thinking, DeepSeek R1, and Claude Opus 4.6) recorded equal or higher best outcome rates under the unstructured prompt. Claude Opus 4.6 Thinking achieved identical best outcome rates under both conditions but lost zero games under the unstructured prompt versus 20 percent under the structured prompt. Only GPT OSS 120b showed a substantial improvement under the structured prompt, doubling its best outcome rate from 30 percent to 60 percent.

Model	Best Outcome % (S)	Best Outcome % (U)	Loss % (S)	Loss % (U)	Avg Turns (S)	Avg Turns (U)	Favors
ChatGPT Thinking	10%	30%	80%	50%	6.7	7.8	U
Claude Opus 4.6	30%	30%	30%	60%	8.3	7.4	Tie
Claude Opus 4.6 Thinking	60%	60%	20%	0%	8.05	8.8	U
Llama 4 Scout	0%	0%	100%	100%	4.4	5.6	Tie
DeepSeek R1	30%	50%	20%	20%	7.6	8.4	U
GPT OSS 120b	60%	30%	20%	35%	8.1	7.8	S

Table 2. Structured (S) versus unstructured (U) prompt performance for the six Stage 2 models. "Favors" indicates the prompt type under which the model achieved the higher best outcome rate.

The ChatGPT Thinking result is particularly striking: under the structured prompt the model lost 80 percent of its games, but under the unstructured prompt its loss rate fell to 50 percent and its draw rate rose from 20 percent to 50 percent. DeepSeek R1 showed a similar but smaller pattern, improving its best outcome rate from 30 percent to 50 percent when the prompt was simplified. Claude Opus 4.6 Thinking behaved distinctively under the unstructured prompt: it generated a JavaScript React component that implemented Tic-Tac-Toe as an embedded interactive application and played through that interface rather than in natural language, achieving a 90 percent draw rate and a 10 percent win rate with zero losses. This behavior represents a qualitatively different use of the unstructured prompt and suggests that for some models the freedom to restructure the task itself may be more valuable than any procedural scaffolding a prompt can provide.

3.3 Opening Move Behavior

Analysis of the move list data reveals that opening move selection is heavily biased toward the center square across nearly all models (Table 3). Five models — ChatGPT Thinking, DeepSeek R1, GPT OSS 120b, Qwen3 Coder Next, and Claude Opus 4.5 — played the center as their first move in 100 percent of the games in which they moved first. ChatGPT Instant and Claude Opus 4.6 each deviated once out of eleven and ten first-move opportunities respectively. No model ever opened with an edge square, which is consistent with the known theoretical weakness of edge openings in Tic-Tac-Toe.

Model	LLM-first Games	Center	Corner	Edge	Overall Loss %
ChatGPT Instant	11	91%	9%	0%	90%
ChatGPT Thinking	10	100%	0%	0%	65%

Model	LLM-first Games	Center	Corner	Edge	Overall Loss %
Gemini 3 Thinking	4	75%	25%	0%	20%
Claude Opus 4.5	5	100%	0%	0%	50%
Claude Opus 4.6	10	90%	10%	0%	45%
Claude Opus 4.6 Thinking	15	60%	40%	0%	13%
Llama 4 Scout	10	50%	50%	0%	100%
DeepSeek R1	9	100%	0%	0%	20%
GPT OSS 120b	13	100%	0%	0%	30%
Qwen3 Coder Next	4	100%	0%	0%	80%

Table 3. Opening move distribution for games in which the model moved first. Overall loss percentage is reproduced from Table 1 for convenient comparison.

Two models showed notable deviation from this pattern. Claude Opus 4.6 Thinking opened with a corner in 40 percent of its first-mover games (6 of 15) and with the center in only 60 percent, while still achieving the lowest loss rate (13%) in the study. This suggests that its corner openings were deliberate strategic choices rather than random deviation, consistent with the fact that both center and corner openings are winning lines for the first player under perfect play. Llama 4 Scout, by contrast, opened with a corner in 50 percent of its first-mover games but lost 100 percent of its twenty total games. The opening-move variation in Llama's play should be interpreted not as strategic sophistication but as an inability to consistently retrieve a dominant opening, consistent with its broader pattern of state-tracking failures described in Section 3.5.

3.4 Best Outcome by First-Mover Status

Breaking the best outcome metric down by first-mover status reveals a consistent asymmetry. Every Stage 2 model that achieved any best outcomes at all achieved substantially more of them when moving second than when moving first (Table 4). Claude Opus 4.6 Thinking achieved a 93 percent best outcome rate when moving second but only 27 percent when moving first. GPT OSS 120b, DeepSeek R1, Claude Opus 4.6, and ChatGPT Thinking all show gaps of forty percentage points or more in the same direction.

Model	Best Outcome % (LLM first)	Best Outcome % (User first)	Difference	Theoretical Ceiling
ChatGPT Thinking	0% (0/10)	40% (4/10)	-40 pp	Win only / Draw or Win
Claude Opus 4.6	0% (0/10)	60% (6/10)	-60 pp	Win only / Draw or Win

Model	Best Outcome % (LLM first)	Best Outcome % (User first)	Difference	Theoretical Ceiling
Claude Opus 4.6 Thinking	27% (4/15)	93% (14/15)	-66 pp	Win only / Draw or Win
Llama 4 Scout	0% (0/10)	0% (0/10)	0 pp	Win only / Draw or Win
DeepSeek R1	11% (1/9)	64% (7/11)	-53 pp	Win only / Draw or Win
GPT OSS 120b	8% (1/13)	65% (11/17)	-57 pp	Win only / Draw or Win

Table 4. Best outcome rate broken down by first-mover status for Stage 2 models. "pp" denotes percentage points. The theoretical ceiling column notes the ceiling for each position under perfect play.

This pattern is a direct consequence of how the best outcome metric is defined rather than a surprising property of model behavior. A second-mover achieves a best outcome by forcing a draw, which under the rules of Tic-Tac-Toe is the ceiling for the second player under perfect play. A first-mover achieves a best outcome only by winning outright, since a draw from the first-mover position means the player failed to convert a theoretical advantage. Because wins require a blunder from the opposing player — perfect play by both sides ends in a draw — the best outcome metric is structurally easier to satisfy from the second-mover position. Defending reactively against opponent threats, which is what second-mover play consists of, is also a tactically simpler task than constructing a forcing sequence from the first move, requiring only that the model recognize and block immediate threats rather than plan multiple moves ahead.

The asymmetry in Table 4 therefore reflects two compounding factors. First, the metric grants a best outcome for draws-as-second-mover but not draws-as-first-mover, biasing the denominator in favor of second-mover games. Second, even without this metric asymmetry, reactive play is easier than proactive play on Tic-Tac-Toe, and the data in Section 3.5 support the interpretation that LLMs in particular struggle more with the middle-game planning required to convert first-move advantage than with the immediate-threat blocking required in second-mover play. The combination of these effects produces the consistent gap visible across models.

3.5 Blunder Timing

Table 5 reports the distribution of blunders across the game. For every model, the overwhelming majority of blunders occurred in turns 4–6, the middle phase of the game. Across all ten models combined, 75 of the 88 blunder-containing games (85 percent) had their blunder in turns 4–6, with only 6 blunders in turns 1–3 and 7 in turns 7–9. The strongest models (Claude Opus 4.6 Thinking, DeepSeek R1, Gemini 3 Thinking) had all their few blunders concentrated entirely in the middle game, with none in the opening or endgame. Only Llama 4 Scout, which frequently misdeclared winners after two or three turns, had a meaningful proportion of early blunders.

Model	Blunder Games	Turns 1–3	Turns 4–6	Turns 7–9
ChatGPT Instant	20	0 (0%)	17 (85%)	1 (5%)
ChatGPT Thinking	14	1 (7%)	12 (86%)	2 (14%)
Gemini 3 Thinking	2	0 (0%)	2 (100%)	0 (0%)
Claude Opus 4.5	5	0 (0%)	4 (80%)	1 (20%)
Claude Opus 4.6	9	0 (0%)	8 (89%)	1 (11%)
Claude Opus 4.6 Thinking	4	0 (0%)	4 (100%)	0 (0%)
Llama 4 Scout	20	4 (20%)	15 (75%)	1 (5%)
DeepSeek R1	1	0 (0%)	1 (100%)	0 (0%)
GPT OSS 120b	7	0 (0%)	7 (100%)	0 (0%)
Qwen3 Coder Next	6	1 (17%)	5 (83%)	0 (0%)

Table 5. Distribution of blunders across the three phases of the game. Percentages within a row sum to at least 100 percent because a single game may be coded as having blunders in multiple phases.

This concentration in the middle game is consistent with a depth-limited lookahead interpretation of model failures. The opening phase has a small set of well-known correct responses (center or corner), and the endgame phase has few remaining legal moves. The middle game is where genuine planning matters: a move on turn 5 often has to account for what the opponent can force on turn 6 and turn 7. The uniformity of this pattern across models of very different architectures and training regimes suggests that middle-game planning failure is a general limitation of current LLMs on short-horizon strategic tasks rather than a property of any particular model.

3.6 Common Failure Modes

Across all models, the same categories of error recurred. Missed blocks were the dominant failure: a model would decline to occupy the square that prevents the opponent from completing a three-in-a-row on the next turn, typically in the middle game as described above. Fork mismanagement was the second major category. Several models, including DeepSeek R1 and GPT OSS 120b, failed to prevent the user from establishing a fork even when the fork setup was visible two moves in advance. A third category involved state-tracking errors. Llama 4 Scout, in particular, repeatedly declared game outcomes on the basis of three symbols in a row without distinguishing whether those symbols belonged to the same player, and in several games declared a winner after only two or three total moves. Finally, hallucinated win declarations appeared in multiple models. Claude Opus 4.6 declared itself the winner on a subsequent turn after failing to detect a user's win, and Qwen3 Coder Next added unauthorized extra moves to claim victory in games that had in fact ended in a draw or a loss.

3.7 Response Time

Response time varied by orders of magnitude across models. Quick-response models such as ChatGPT Thinking and the Claude Opus variants typically replied within a few seconds per turn. DeepSeek R1 frequently required fifteen to thirty seconds of reasoning per turn under the structured prompt, and in one Stage 1 session the model continued reasoning for over fifteen minutes without producing a response before the session was terminated. Chat-log inspection revealed that DeepSeek R1 was not stalling but was instead enumerating possible game progressions and board representations before committing to a first move, without soliciting user input. This behavior illustrates a potential cost of procedural prompts: a model instructed to check every condition may attempt to do so exhaustively at the cost of ever completing a turn. Because wall-clock time also partially captures human reaction time, the recorded game times are more accurate for slower models (where human reaction time is a smaller fraction of total time) than for fast models, which limits their value as a pure measure of model speed.

4. Discussion

Three findings from this study have implications that extend beyond the specific comparison of structured and unstructured prompts. The first is the non-uniform and sometimes negative effect of structured prompting. This result contrasts with the common prescription in prompt engineering guidance, which treats explicit, decomposed instruction as a general-purpose tool for improving reasoning quality. A plausible explanation is that the overhead of executing the structured prompt's checking procedure competes with the resources the model would otherwise allocate to forward-looking move selection. Models instructed to enumerate every row, column, and diagonal on every turn may succeed in reporting the current board state accurately but fail to simulate the opponent's next move, which is the reasoning step that actually determines whether a block or fork is required. The result for GPT OSS 120b, which did benefit from structured prompting, suggests that some models are configured in a way that rewards explicit scaffolding; the structured prompt may function less as a reasoning aid and more as a signal about the seriousness of the task.

The second finding — that every Stage 2 model performed better as second-mover than as first-mover — has a straightforward two-part explanation. Part of the gap is structural: the best outcome metric credits draws from the second-mover position but not from the first-mover position, which mechanically biases the metric in favor of second-mover games whenever any game ends in a draw. The remaining gap reflects a genuine asymmetry in the underlying task. Second-mover play in Tic-Tac-Toe is primarily reactive — the model must recognize an immediate threat and block it — while first-mover play requires constructing a forcing sequence across multiple moves. The blunder-timing data in Section 3.5 indicate that LLMs fail most often in the middle game, which is precisely where first-mover construction demands the most planning. One implication for prompt design is that scaffolding meant to encourage look-ahead may need to be qualitatively different when the model has initiative (and must construct threats) versus when it has none (and must merely defend against them).

The third finding — that 85 percent of blunders across all models occur in turns 4–6 — supports a depth-limited search interpretation. Models appear capable of responding to immediate threats visible in the current board state but struggle when the threat requires simulating one or two moves ahead. Fork failures, which require recognizing that a single opponent move will create two unblockable threats, represent a harder version of the same problem and are accordingly common among the mid-game blunders. The fact that this pattern holds across architectures as different as Llama 4 Scout and Claude Opus 4.6 Thinking suggests it is a property of how current LLMs represent short-horizon game states, rather than an idiosyncrasy of any particular training regime.

Finally, the behavior of Claude Opus 4.6 Thinking under the unstructured prompt — recasting the task as an embedded interactive application — raises a broader question about what "playing Tic-Tac-Toe" means for a generative model. When freed from the constraint of playing turn by turn in natural language, the model chose an interface better matched to its actual capabilities. This suggests that some of the failure modes observed across other models may reflect a mismatch between the natural-language chat format and the structural demands of board-game play, rather than a fundamental limitation of the underlying reasoning ability.

5. Limitations

Several factors limit the scope of the conclusions that can be drawn from this study. Sample sizes are small. Ten games per prompt condition per model is sufficient to establish broad patterns but not to support fine-grained statistical comparison, and no confidence intervals or significance tests were computed. The study also involved a single human opponent; although the opponent played for optimal outcomes, their play was not independently validated against an algorithmic benchmark, and variation in their attention or consistency across sessions contributes uncontrolled noise.

Model identity and version are also difficult to fix. Commercial LLMs are updated without public notice, and some of the tested models expose only their outputs, not the underlying weights. A performance difference observed today may not replicate six months later if the provider silently updates the model. There is also the possibility that some models are deliberately configured to avoid decisively defeating users in casual tasks; the extent to which such safety or engagement considerations affect observed play cannot be determined from outside.

Prompt variation across models introduces another confound. Although both prompt conditions follow a common template (see Appendix A), minor wording differences were retained when a particular prompt failed to elicit engaged play from a given model. The most notable deviations are the structured prompt for DeepSeek R1, which used 1–9 numbering rather than (row, column) coordinates and omitted the diagonal enumeration clause; and three variants of the unstructured prompt ranging from six words to over sixty. Findings should therefore be interpreted as evidence about classes of prompting approach rather than about specific word-for-word prompts.

First-mover assignment was not controlled, and the resulting LLM-first and user-first game counts are uneven across models (Table 4). The first-mover paradox finding is robust to this imbalance because every non-Llama model shows a large negative gap, but precise quantification would require a balanced design. Finally, Tic-Tac-Toe is a simple and solved game. The failure modes observed here may not generalize to longer-horizon strategic games with larger state spaces, and the patterns of prompt sensitivity observed here may not transfer to tasks outside the game domain.

6. Conclusion

This study finds that structured prompting produces inconsistent and frequently negative effects on LLM performance in Tic-Tac-Toe. Only one of six tested models clearly benefited from a structured prompt, while several performed measurably worse when asked to follow an explicit procedural checklist. Two additional findings emerged from the move-level data: every non-trivial model achieved substantially better outcomes as second-mover than as first-mover, a pattern driven by the definition of the best outcome metric and by the tactical simplicity of reactive defense relative to first-mover offense; and roughly 85 percent of all blunders concentrated in the middle phase of the game, consistent with a depth-limited lookahead interpretation of LLM reasoning. The most effective prompting approach appears to depend on the model's underlying architecture and on whether the model is able to reframe the task itself, as Claude Opus 4.6 Thinking did by generating an embedded interactive interface. These findings suggest that prompt engineering for strategic reasoning should be treated as an empirical, per-model exercise rather than as the application of general best practices, and that simple benchmarks like Tic-Tac-Toe retain substantial diagnostic value for probing the boundaries of LLM reasoning capability.

7. Directions for Strengthening This Research

Several additions would meaningfully increase the statistical power, validity, and interpretive depth of the present study. They are organized below by type of improvement, with items revised to reflect the current data in hand.

7.1 Decompose the First-Mover Effect

The substantial best outcome gap between first-mover and second-mover games (Section 3.4) reflects two compounding factors: an asymmetry in the metric itself, and a genuine tactical asymmetry between constructing threats and defending against them. A follow-up study could separate these factors by reporting alternative metrics alongside best outcome, such as move-quality score against a minimax reference, which would measure how close each individual move was to optimal regardless of eventual outcome. Complementary analysis of reasoning traces for models that expose their chain-of-thought would clarify whether LLMs treat first-move and second-move positions differently — for example, whether they plan forward from the first move or simply pattern-match to a common opening.

7.2 Increase Statistical Power

Ten games per prompt condition per model is the minimum at which meaningful effects can be suggested but not the minimum at which they can be confirmed. A reasonable target would be thirty to fifty games per condition per model, which would permit the calculation of confidence intervals on best outcome rates and the use of standard hypothesis tests (chi-square for categorical outcomes, Mann-Whitney for turn counts) to determine whether observed differences between prompt conditions are statistically significant.

7.3 Introduce an Algorithmic Opponent

The single-human-opponent design, though defended by the opponent's commitment to optimal play, remains an uncontrolled source of variance. Replacing the human with a deterministic algorithmic opponent — most naturally a minimax player, with optional noise injected for variety — would remove opponent variance entirely and make every game exactly reproducible. This would also permit much larger sample sizes, since a human is not needed in the loop. A combined design in which each model plays a fixed set of algorithmic opponents and also a small number of human games would retain the ecological validity of human play while providing a high-precision reference point.

7.4 Control Move Order

First-mover status was recorded but not controlled, and the unequal LLM-first and user-first game counts visible in Table 4 reflect this. Forcing a balanced split — five LLM-first and five user-first games within each prompt condition — would remove this source of variation and make results across models directly comparable. Given the size of the first-mover paradox effect, balancing move order is now arguably the most impactful methodological improvement available.

7.5 Standardize Prompts Across Models

The current study permits some prompt wording variation across models. This is pragmatically useful for maximizing model engagement but confounds the prompt-type comparison. A complementary study using two fixed prompt templates verbatim across every model — accepting that some models will engage less well with the fixed versions — would isolate the effect of prompt type from the effect of prompt adaptation. A useful follow-up design would additionally introduce several distinct structured prompts and several distinct unstructured prompts per model, treating prompt wording as a crossed factor rather than a nuisance.

7.6 Expand the Metric Set

The study could productively add several quantitative measures. A move-quality score, computed by comparing each model move against the optimal move determined by minimax, would reveal not only whether the model won or lost but how close to perfect play each individual move was. Tracking the turn at which the first suboptimal move occurred — distinguished from the turn at which the decisive blunder occurred — would clarify whether models drift gradually from optimal play or fail suddenly. Counting hallucinated moves, invalid move attempts, and incorrect win declarations as separate error types would permit finer diagnostic analysis than the current binary

blunder flag. Finally, categorizing blunders (missed block, missed win, failed fork defense, failed fork creation) would clarify which specific reasoning steps are most failure-prone.

7.7 Decompose the Structured Prompt

The current study contrasts only two prompt extremes. An incremental design that adds one structural element at a time — coordinates only, coordinates plus board reconstruction, coordinates plus reconstruction plus win-check enumeration — would identify which specific components of the structured prompt help or hurt, rather than treating the structured prompt as an indivisible unit. This decomposition is essential for producing actionable prompt-engineering guidance from the results.

7.8 Extend to Richer Games

Tic-Tac-Toe bounds the reasoning horizon at nine moves, which may conceal capability differences that only manifest over longer play. Replicating the study on Connect Four, Gomoku, or small-board chess puzzles would test whether the observed prompt-sensitivity and first-mover findings hold as strategic depth increases. These games retain clear win conditions and moderate state spaces while demanding genuine lookahead.

7.9 Test the Code-Generation Workaround

Claude Opus 4.6 Thinking's spontaneous generation of a React application to play Tic-Tac-Toe suggests that prompting models to write code, rather than play turn-by-turn, may circumvent the chat-format limitations responsible for many observed failures. A targeted experiment in which every model is explicitly instructed to write a Tic-Tac-Toe engine and then play against its own engine would test whether the problem lies in the game or in the interaction format. This framing reinterprets "playing Tic-Tac-Toe" as a code-generation task, an area of demonstrated LLM strength, and could yield substantially higher quality play without any change in underlying reasoning capability.

7.10 Formal Version Control

Recording the exact model version, API endpoint, temperature setting, and date of each game would allow future researchers to replicate the study under matched conditions and would make performance regressions or improvements across model updates observable. Including a sanity-check game at the start and end of each testing session, in which the model is asked to solve a known Tic-Tac-Toe position, would detect whether model behavior drifts during the course of data collection.

Appendix A. Verbatim Prompts

The prompts below are reproduced verbatim from the chat logs of each session. Minor wording differences across models reflect the study's design choice to use the prompt variant that best engaged each model rather than enforcing strict verbatim consistency.

Standard Structured Prompt (used with minor variation for most models)

Play Tic Tac Toe, but follow these rules strictly: Represent the board with coordinates (rows 1-3 bottom to top, columns 1–3 left to right). After every move: Reconstruct the board from the full move list (no assumptions). Before making your move: Check ALL win conditions explicitly: a. Horizontal (every row) b. Vertical (every column) c. Diagonal ↗ and ↘ (enumerate each sequence of 3) List every 3-cell diagonal you check and state whether it is a win. Only proceed if you confirm no player has already won. If a win exists, stop immediately and declare the winner. Do not skip steps. Your goal is to win with perfect play.

Used with: ChatGPT Instant, ChatGPT Thinking, Claude Opus 4.6 Thinking, Llama 4 Scout, GPT OSS 120b, Qwen3 Coder Next. Claude Opus 4.6 received the same prompt with the additional sentence "Provide a move summary after each game."

DeepSeek R1 Structured Prompt

Play Tic Tac Toe against me, but follow these rules strictly: Represent the board with numbers 1-9. After every move: Reconstruct the board from the full move list (no assumptions). Before making your move: Check ALL win conditions explicitly: a. Horizontal (every row) b. Vertical (every column) c. Diagonal ↗ and ↘. Only proceed if you confirm no player has already won. If a win exists, stop immediately and declare the winner. Do not skip steps. Your goal is to win with perfect play.

Used with: DeepSeek R1. Differs from the standard structured prompt in using 1–9 square numbering rather than (row, column) coordinates and in omitting the diagonal enumeration instruction.

Minimal Unstructured Prompt

play tic tac toe against me

Used with: ChatGPT Thinking, Claude Opus 4.6 Thinking, Llama 4 Scout, GPT OSS 120b.

Extended Unstructured Prompt

act as a tic tac toe player. You are playing tic tac toe against me, represent the board as a 3 by 3 grid with each square labeled 1-9. Check each win condition after your turn and stop if a winner is found. Your goal is to win with perfect play.

Used with: Claude Opus 4.5, Claude Opus 4.6. DeepSeek R1 received a version omitting the final "Your goal is to win with perfect play" sentence.

Qwen3 Coder Next Unstructured Prompt

play tic tac toe against me, your goal is to win with perfect play